Cornell Bowers CIS
Computer Science

Applied High-Performance and Parallel Computing

Parallel Matrix Multiply

Professor Giulia Guidi

03/05/2026

A number of slides were taken from or inspired by slides by Anthony Nouy and James Demmel

Dense Linear Algebra Beyond MatMul

- Linear Systems: $Ax = b$

  The bread and butter of numerical linear algebra; on size and structure. Direct methods and iterative methods depending on size and structure.

- Least Squares: Choose x to minimize $||Ax-b||_2$ (or $||Ax-b||_2^2 + ||x||_2^2$)
  - Overdetermined or underdetermined; Unconstrained, constrained, weighted (or ridge)
- Eigenvalues and vectors of Symmetric Matrices
  - Standard ($Ax = \lambda x$), Generalized ($Ax=\lambda Bx$)
- Eigenvalues and vectors of Unsymmetric matrices
  - Eigenvalues, Schur form, eigenvectors, invariant subspaces
  - Standard, Generalized

from "nicest" (linear systems, structured eigenproblems) to "hardest" (unsymmetric generalized eigenproblems)

Dense Linear Algebra Beyond MatMul

- Singular values and vectors (SVD)
  - Standard, Generalized
- Different matrix structures – 28 types in LAPACK
  - Real, complex; symmetric, Hermitian, positive definite; dense, triangular, banded, etc.

Exploiting structure isn't *cosmetic*: a banded Cholesky is orders of magnitude cheaper than dense LU.

Deterministic Linear Algebra

I.e., classic

- Linear algebra is at the core of scientific computing, statistics, data analysis, AI
- Classical linear algebra methods require elementary algebraic operations on matrices and vectors
  - E.g., norm, dot product, matrix-vector product, matrix-matrix product

Deterministic Linear Algebra

GEMM = Generalized Matrix Multiplication

- Linear algebra is at the core of scientific computing, statistics, data analysis, AI
- Classical linear algebra methods require elementary algebraic operations on matrices and vectors
  - E.g., norm, dot product, matrix-vector product, matrix-matrix product

If you can cast your algorithm in terms of GEMM, you get hardware efficiency almost for free. This is why deep learning frameworks are fast, and why so much of HPC algorithm design is about restructuring computations to maximize GEMM proportion. The sparse world (SpMV, SpMM) is harder precisely because the primitives are less regular **(yet, I try!)**.

- GEMM is regular: hardware and compiler love it!

Randomized Numerical Linear Algebra (RandNLA)

- Randomized linear algebra aims at
  - **Reducing the complexity** of algorithms,

Classical algorithms like Gaussian elimination are **$O(n^3)$**; randomized methods (e.g., randomized SVD, sketch-and-solve) can reduce this to **$O(n^2k)$** or even **$O(nnz \cdot k)$** for rank-k approximations, which is key at scale

Source: https://anthony-nouy.github.io/tutorials/morss/morss_2019_rla.pdf

Randomized Numerical Linear Algebra (RandNLA)

- Randomized linear algebra aims at
  - **Reducing the complexity** of algorithms,
  - **Improving stability**,

Classical algorithms can sometimes be numerically unstable (e.g., normal equations for least squares); randomized preconditioning or sketch-based approaches can improve conditioning, not just approximate the answer.

Source: https://anthony-nouy.github.io/tutorials/morss/morss_2019_rla.pdf

Randomized Numerical Linear Algebra (RandNLA)

- Randomized linear algebra aims at
  - **Reducing the complexity** of algorithms,
  - **Improving stability**,
  - Taking into account **computational constraints** (data not available from RAM, streamed data),

You can compress on the fly using sketching based appraoches

Source: https://anthony-nouy.github.io/tutorials/morss/morss_2019_rla.pdf

Randomized Numerical Linear Algebra (RandNLA)

- Randomized linear algebra aims at
  - **Reducing the complexity** of algorithms,
  - **Improving stability**,
  - Taking into account **computational constraints** (data not available from RAM, streamed data),
  - Or fully exploiting **modern computational architectures** (parallel computing, cloud computing)

Randomized algorithms are sometimes described as "turning sparse problems into dense problems" at the sketch level. You pay an initial cost to compress, then subsequent work operates on the small, dense, k-dimensional sketch, which is ideal for modern hardware.

Source: https://anthony-nouy.github.io/tutorials/morss/morss_2019_rla.pdf

Randomized Numerical Linear Algebra (RandNLA)

- Can I always use RandNLA?
  - It requires **approximate low-rank structure** in the matrices;
  - It requires that you can tolerate probabilistic guarantees;

Source: https://anthony-nouy.github.io/tutorials/morss/morss_2019_rla.pdf

Randomized Numerical Linear Algebra (RandNLA)

- Can I always use RandNLA?
  - It requires **approximate low-rank structure** in the matrices;
  - It requires that you can tolerate probabilistic guarantees;
  - It doesn't help when the bottleneck isn't arithmetic;

Source: https://anthony-nouy.github.io/tutorials/morss/morss_2019_rla.pdf

Randomized Numerical Linear Algebra (RandNLA)

- Can I always use RandNLA?
  - It requires **approximate low-rank structure** in the matrices;
  - It requires that you can tolerate probabilistic guarantees;
  - It doesn't help when the bottleneck isn't arithmetic;

- RandNLA is well-suited when your problem is:
  - Large, approximately low-rank, and you want an approximate answer fast;
- It's poorly suited when your problem is:
  - Well-conditioned and full-rank, small, requires exact answers, or has structure that randomization destroys.

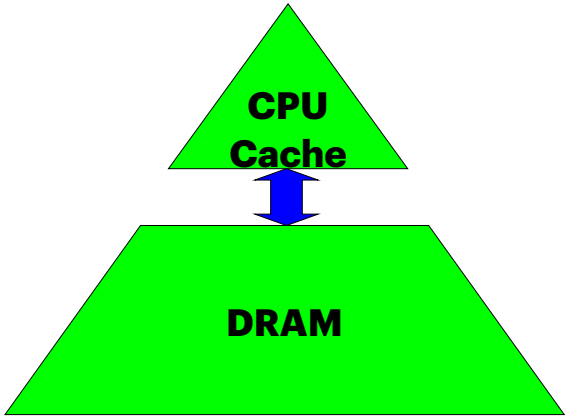
- RandNLA not covered further in this class but could be a **class project**, if interested

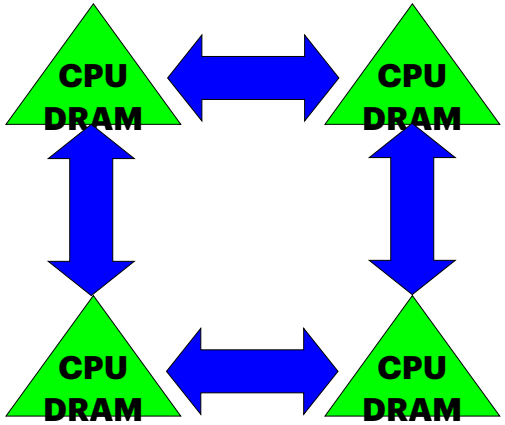
Source: https://anthony-nouy.github.io/tutorials/morss/morss_2019_rla.pdf

Parallel Generalized Matrix Multiply (GEMM)

Let's Avoid Communication 1/3

- Any algorithm has:
  - The **arithmetic cost (FLOPS)**
  - The **communication cost,** i.e., moving data between
    - Levels of a memory hierarchy (sequential or shared memory parallelism case)
    - Processors over a network (distributed memory parallelism case)



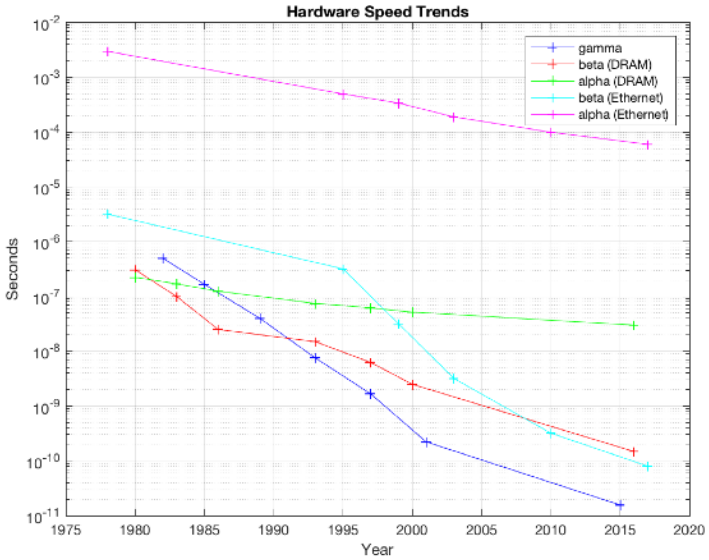


Let's Avoid Communication 2/3

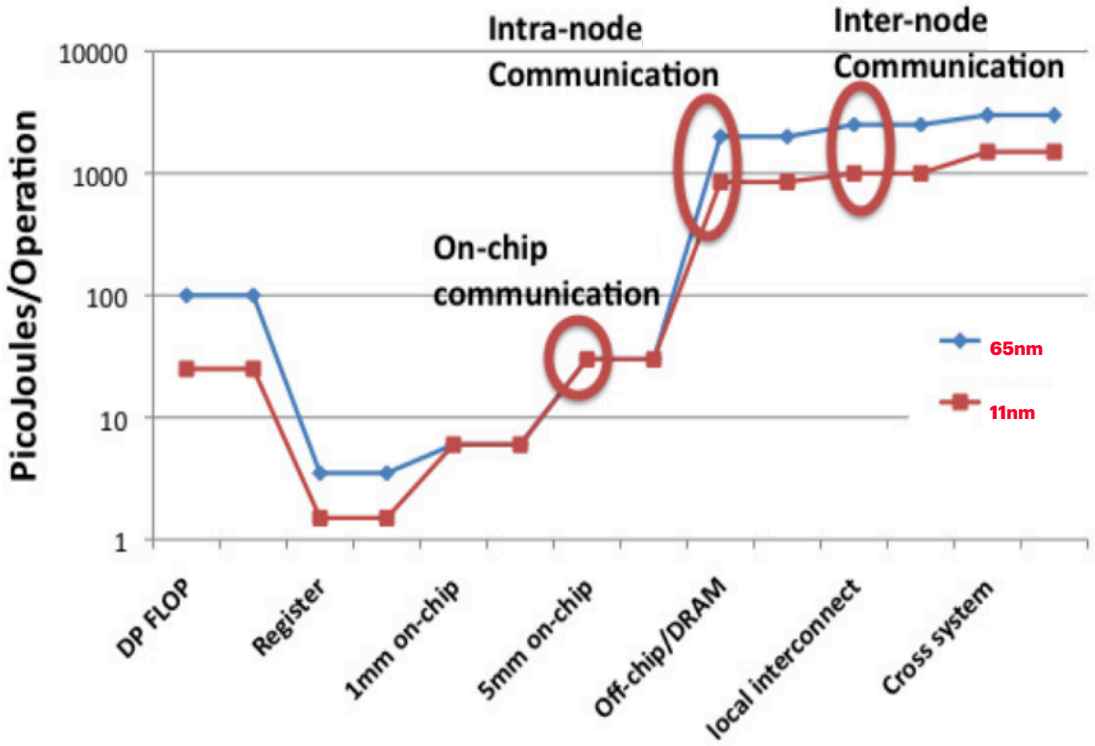
- The running time of an algorithm is sum of 3 terms:
  - # flops * time_per_flop
  - # words moved / bandwidth
  - # messages * latency

  } **Communication**

- time_per_flop (γ) << 1/bandwidth (β) << latency (α)
- The gap is growing exponentially with time



Let's Avoid Communication 3/3



Source: John Shalf, LBL

Goal: Organize LA to Avoid Communication

- Between all memory hierarchy levels
  - L1, L2, DRAM, network, etc.
- The goal is to avoid communication, not just hiding it (overlap with arithmetic)
  - The speedup ≤ 2x
- So that arbitrary speedup/energy saving is possible
  - Lots of open problems!

Review: Blocked Matrix Multiply

Classic example: **Matrix Multiply**

- Load b×b block of A
- Load b×b block of B
- Compute product of blocks
- Finally, accumulate into b × b block of C

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} & b_{13} & b_{14} \\ b_{21} & b_{22} & b_{23} & b_{24} \\ b_{31} & b_{32} & b_{33} & b_{34} \\ b_{41} & b_{42} & b_{43} & b_{44} \end{bmatrix}$$

$$+ \begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} & b_{13} & b_{14} \\ b_{21} & b_{22} & b_{23} & b_{24} \\ b_{31} & b_{32} & b_{33} & b_{34} \\ b_{41} & b_{42} & b_{43} & b_{44} \end{bmatrix}$$

$$= \begin{bmatrix} c_{11} & c_{12} & c_{13} & c_{14} \\ c_{21} & c_{22} & c_{23} & c_{24} \\ c_{31} & c_{32} & c_{33} & c_{34} \\ c_{41} & c_{42} & c_{43} & c_{44} \end{bmatrix}$$

Can **improve data locality** but must be tuned for your CPU (i.e., your cache sizes)

Picture from https://marek.ai/matrix-multiplication-on-cpu.html

Review: Blocked Matrix Multiply

The tile structure that helps cache also defines the parallel decomposition

- Break A^{nxn}, B^{nxn},
- b chosen so that 3 b-by-b blocks fit in cache

```
for i = 1 to n/b
  for j = 1 to n/b
    for k = 1 to n/b
      C(i, j) = C(i, j) + A(i, k)·B(k, j) // b-by-b mat
```

- If b = 1, i.e., "naïve" algorithm, we want b larger
  - $(n/b)3 \cdot 4b2 = 4n3/b$ reads/writes altogether
  - It's minimized when $3b^2$ = cache size = M, yielding $O(n^3/M^{1/2})$ reads/writes
- But what if we had more levels of memory? (L1, L2, cache etc)?
  - It would need 3 more nested loops per level
  - Or recursive (cache-oblivious algorithm) also possible

C, you need a row
mn block **b** of B. If you
have M words of fast memory, the question
is: *What's the largest useful chunk of computation you can do before you need to go back to slow memory?*

$b^3 / b^2 = b \approx \sqrt{M}$

Because you perform b^3 flops but only move $3b^2 \approx M$ words

Communication Lower Bound

If we assume n^3 algorithm:

- The sequential case:
  - The lower bound on #words moved to/from slow memory = $\Omega(n^3/M^{1/2})$ [Hong, Kung, 1981]
  - It is **attained using blocked or cache-oblivious algorithm**
- The parallel case on P processes:
  - Let M be memory per processor; we assume it's load balanced (one proc doing all the work is slow)
  - Lower bound on #words moved = $\Omega((n^3/P)/M^{1/2})$ [Irony, Tiskin, Toledo, 2004]
  - If $M = 3n^2/P$ (one copy of each matrix), then **lower bound = $\Omega(n^2/\sqrt{P})$**
  - It is **attained by SUMMA**, Cannon's algorithm

Common Parallel Decomposition

Owner-Computes rule: each processor owns a block of C and computes it.

1D Row Layout

- It's simple to implement
- Each proc needs full A and B
- $O(n^2)$ data per process
- Doesn't scale

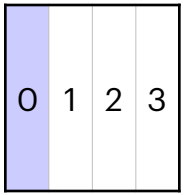
2D Block Layout ← Better!

$\sqrt{p} \times \sqrt{p}$ process grid

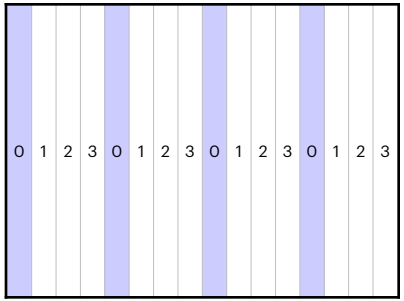
- Each proc needs only a row-panel of A and column-panel of B
- Data per proc: $O(n^2/\sqrt{P})$
- The foundation for Cannon & SUMMA

Drawing made by Claude AI

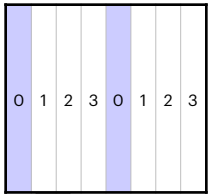
Different Parallel Data Layout for Matrices



1) 1D Column Blocked Layout



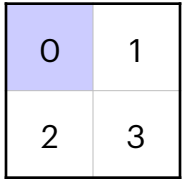
2) 1D Column Cyclic Layout



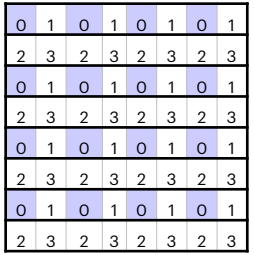
b

3) 1D Column Block Cyclic Layout

4) Row versions of the previous layouts



5) 2D Row and Column Blocked Layout



Generalizes others

6) 2D Row and Column Block Cyclic Layout

Parallel GEMM Design and Performance Predictions

- Computing C = C + A*B
- Using basic algorithm: $2n^3$ FLOPS
- The variables are:
  - Data layout: 1D? 2D? Other? [1D Ring won't hit communication lower bound]
  - Topology of machine: Ring? Torus? [A torus matches Cannon's algorithm, ring favors broadcast-based algorithms like SUMMA]
  - Scheduling communication [Overlap communication and computation]

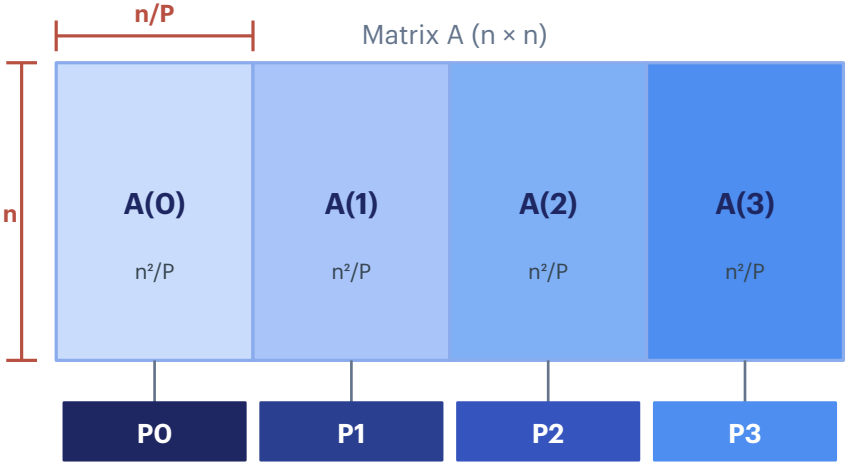
- Use of performance models for algorithm design
  - message time = "latency" + #words * time-per-word = $\alpha + n*\beta$

Send fewer, larger messages

- The efficiency (in any model):
  - Parallel efficiency = serial_time / (P*parallel_time)
  - Perfect (linear) speedup ↔ efficiency = 1

Matrix Multiply with 1D Column Layout

1D Column Distribution



$$C(i) = C(i) + \Sigma_j\, A(j) \cdot B(j, i)$$

$T_{comm} = (P-1)\cdot\alpha + (P-1)\cdot(n^2/P)\cdot\beta \approx P\cdot\alpha + n^2\cdot\beta$

(This assumes ring Allgather.)

Bound Analysis

The words sent per processor:

$w = (P-1) \cdot n^2/P \approx n^2$

Thus, the *entire* matrix A must be received

Hong–Kung lower bound ($M = n^2/P$):

$w = \Omega(\, n^3 / (P \cdot \sqrt{M})\,) = \Omega(\, n^2 / \sqrt{P}\,)$

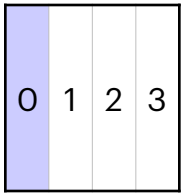
This is the *communication-optimal target*

Gap: n^2 versus $n^2/\sqrt{P}$; there's a **factor of $\sqrt{P}$ overhead**

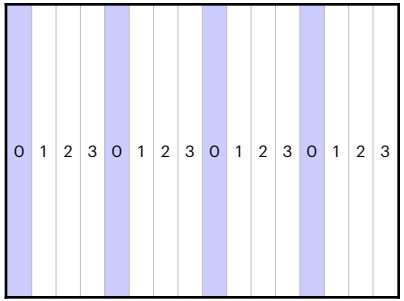
1D <u>cannot</u> achieve communication lower bound

Drawing made by Claude AI

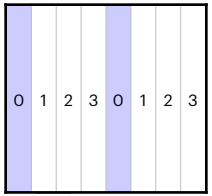
Let’s Try 2D Layout



1) 1D Column Blocked Layout



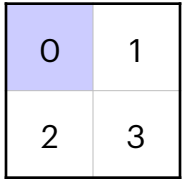
2) 1D Column Cyclic Layout



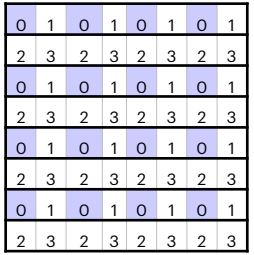
b

3) 1D Column Block Cyclic Layout

4) Row versions of the previous layouts



5) 2D Row and Column Blocked Layout



Generalizes others

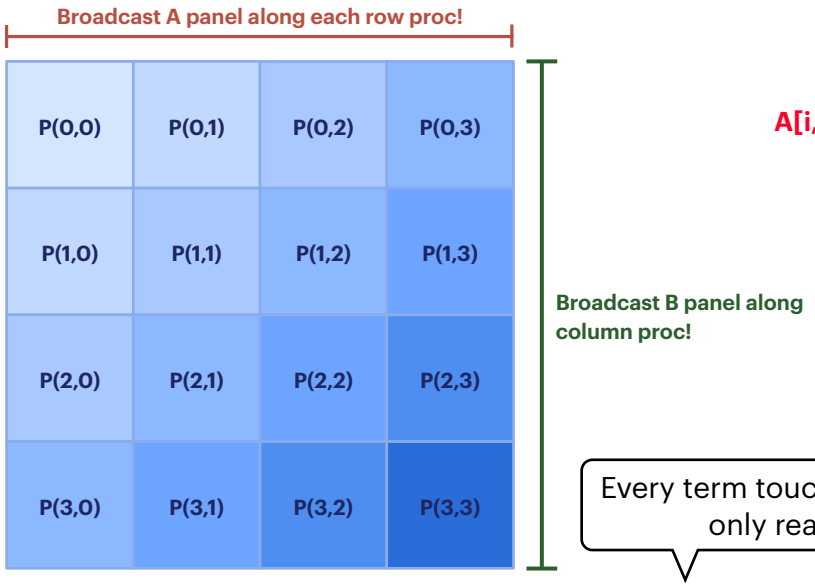
6) 2D Row and Column Block Cyclic Layout

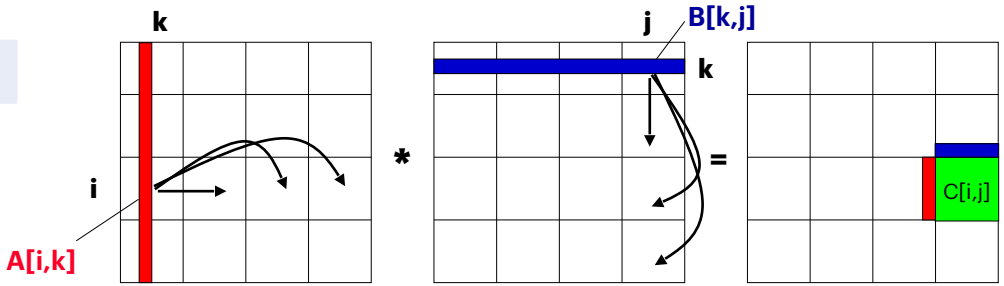
Parallel Matrix Multiply

- SUMMA
  - Scalable Universal Matrix Multiply Algorithm
  - It attains communication lower bounds (within log p)
- Cannon
  - It's historically the first to attain the lower bound
  - **But more assumptions**
    - **A** and **B** square
    - P a perfect square
- 2.5D SUMMA
  - Uses more memory to communicate even less
- Parallel Strassen
  - It attains different, even lower bounds *(not covered this semester)*

SUMMA: Scalable Universal Matrix Multiply Algorithm

2D Processor Grid ($\sqrt{P} \times \sqrt{P}$)





- C[i, j] is $n/P^{1/2}$ x $n/P^{1/2}$ submatrix of C on processor P(i, j)
- A[I, k] is $n/P^{1/2}$ x b submatrix of A
- B[k, j] is b x $n/P^{1/2}$ submatrix of B
- C[i, j] = C[i, j] + Σ_k A[i, k]*B[k, j]
  - It's a summation over submatrices!

Every term touches all of **C**: no output locality, but each term only reads one column of **A** and one row of **B**

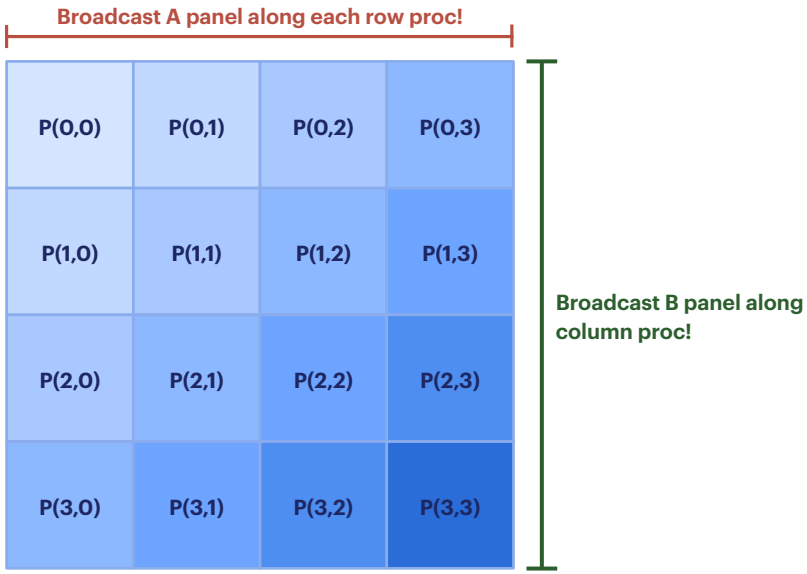
$$C = \Sigma_k A(:, k) \cdot B(k, :)$$

Outer product formulation

Drawing (left) made by Claude AI

SUMMA: Scalable Universal Matrix Multiply Algorithm

2D Processor Grid ($\sqrt{P} \times \sqrt{P}$)



$$C = \Sigma_k A(:, k) \cdot B(k, :)$$

Outer product formulation

Algorithm & Communication

SUMMA steps ($\sqrt{P}$ broadcasts total):

1. Broadcast $n \times n/\sqrt{P}$ panel of A along each row
2. Broadcast $n \times n/\sqrt{P}$ panel of B along each column
3. Each proc: local $C(i,j) \mathrel{+}= A_{panel} \times B_{panel}$

Communication per processor:

$$w \approx \sqrt{P} \cdot 2 \cdot (n^2/P) \approx 2n^2 / \sqrt{P}$$

It matches $\Omega(n^2/\sqrt{P})$ lower bound! It's **communication optimal***.

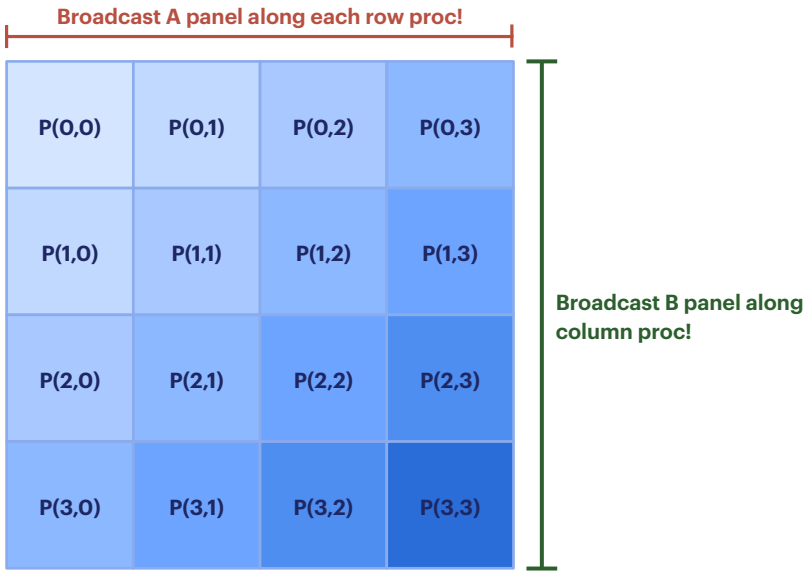
2D grid distributes both A and B broadcasts achieving **$\sqrt{P}$ saving**

***It attains lower bound **within a logP factor** (pretty good!). The logP factor comes from the cost of the broadcast collective that SUMMA uses.

Drawing made by Claude AI

SUMMA: Scalable Universal Matrix Multiply Algorithm

2D Processor Grid ($\sqrt{P} \times \sqrt{P}$)



total #messages total #words

$$T_{comm} = \sqrt{P}\cdot\log P\cdot\alpha + \log P\cdot(n^2/\sqrt{P})\cdot\beta$$

(This assumes Broadcast tree; if using hardware with native broadcast support the logP can be effectively absorbed into a constatnt.)

Algorithm & Communication

SUMMA steps ($\sqrt{P}$ broadcasts total):

1. Broadcast $n\times n/\sqrt{P}$ panel of A along each row
2. Broadcast $n\times n/\sqrt{P}$ panel of B along each column
3. Each proc: local $C(i,j) \mathrel{+}= A_{panel} \times B_{panel}$

Communication per processor:

$$w \approx \sqrt{P} \cdot 2 \cdot (n^2/P) \approx 2n^2 / \sqrt{P}$$

It matches $\Omega(n^2/\sqrt{P})$ lower bound! It's **communication optimal***.

2D grid distributes both A and B broadcasts achieving **$\sqrt{P}$ saving**

***It attains lower bound **within a logP factor** (pretty good!). The logP factor comes from the cost of the broadcast collective that SUMMA uses.

Drawing made by Claude AI

Cannon's Algorithm (1/2): Initial Skew

Before Skew: Natural Layout

	Col 0	Col 1	Col 2
P(0,·)	**A(0,0)** B(0,0)	**A(0,1)** B(0,1)	**A(0,2)** B(0,2)
P(1,·)	**A(1,0)** B(1,0)	**A(1,1)** B(1,1)	**A(1,2)** B(1,2)
P(2,·)	**A(2,0)** B(2,0)	**A(2,1)** B(2,1)	**A(2,2)** B(2,2)

- A(i,j) = block row i, col j of A
- B(i,j) = block row i, col j of B

After Skew: Each Row/Col Offset by Index

	Col 0	Col 1	Col 2	
P(0,·)	**A(0,0)** B(0,0)	**A(0,1)** B(1,1)	**A(0,2)** B(2,2)	
P(1,·)	**A(1,1)** B(1,0)	**A(1,2)** B(2,1)	**A(1,0)** B(0,2)	← 1
P(2,·)	**A(2,2)** B(2,0)	**A(2,0)** B(0,1)	**A(2,1)** B(1,2)	← 2
		↑1	↑2	

- **Row i: A shifted left by i**
- **Col j: B shifted up by j**

Drawing made by Claude AI

Cannon's Algorithm (2/2): Cyclic Shifts & Communication

Algorithm & Communication

Step 0 (after skew)

A(0,0) B(0,0)	**A(0,1)** B(1,1)	**A(0,2)** B(2,2)
A(1,1) B(1,0)	P(1, 1) **A(1,2)** B(2,1)	**A(1,0)** B(0,2)
A(2,2) B(2,0)	**A(2,0)** B(0,1)	**A(2,1)** B(1,2)

→ A ← left, B ↑ up

Step 1

A(0,1) B(1,0)	**A(0,2)** B(2,1)	**A(0,0)** B(0,2)
A(1,2) B(2,0)	P(1, 1) **A(1,0)** B(0,1)	**A(1,1)** B(1,2)
A(2,0) B(0,0)	**A(2,1)** B(1,1)	**A(2,2)** B(2,2)

→ A ← left, B ↑ up

Step 2

A(0,2) B(2,0)	**A(0,0)** B(0,1)	**A(0,1)** B(1,2)
A(1,0) B(0,0)	P(1, 1) **A(1,1)** B(1,1)	**A(1,2)** B(2,2)
A(2,1) B(1,0)	**A(2,2)** B(2,1)	**A(2,0)** B(0,2)

- **Row i: A shifted left by i**
- **Col j: B shifted up by j**

- A(i,j) = block row i, col j of A
- B(i,j) = block row i, col j of B

Each step: C(i, j) += $A_{local} \times B_{local}$, then shift

P(1, 1) accumulates: A(1, 2)·B(2, 1) + A(1, 0)·B(0, 1) + A(1, 1)·B(1, 1) = C(1, 1)

Drawing made by Claude AI

Cannon's Algorithm: Cyclic Shift Matrix Multiply

Initial Skew + Cyclic Shifts

Initial skewed alignment ($\sqrt{P} = 3$)

	Col 0	Col 1	Col 2	
P(0,·)	A(0,0) / B(0,0)	A(0,1) / B(1,1)	A(0,2) / B(2,2)	
P(1,·)	A(1,1) / B(1,0)	A(1,2) / B(2,1)	A(1,0) / B(0,2)	← 1
P(2,·)	A(2,2) / B(2,0)	A(2,0) / B(0,1)	A(2,1) / B(1,2)	← 2
		↑1	↑2	

$$C(i, j) \mathrel{+}= A(i, k) \cdot B(k, j) \quad \text{for } k = 0, \ldots, \sqrt{P}-1$$

Algorithm & Communication

Cannon steps ($\sqrt{P}$ steps total):

1. Initial skew: shift $A(i,\cdot)$ left by i, $B(\cdot,j)$ up by j
2. Repeat $\sqrt{P}$ times:
   - Do local multiply: $C(i,j) \mathrel{+}= A_{local} \times B_{local}$
   - Shift A left by 1 (cyclic along row)
   - Shift B up by 1 (cyclic along col)

Communication per processor:

$$w = 2\sqrt{P} \cdot (n^2/P) = 2n^2/\sqrt{P}$$

The same bandwidth as SUMMA, but **no broadcast needed**

<u>Caveat</u>: **It has more assumptions: A** and **B** must be square and **P** must be a perfect square

Drawing made by Claude AI

Cannon's Algorithm: Cyclic Shift Matrix Multiply

Initial Skew + Cyclic Shifts

Initial skewed alignment ($\sqrt{P} = 3$)

	Col 0	Col 1	Col 2	
P(0,·)	**A(0,0)** / B(0,0)	**A(0,1)** / B(1,1)	**A(0,2)** / B(2,2)	
P(1,·)	**A(1,1)** / B(1,0)	**A(1,2)** / B(2,1)	**A(1,0)** / B(0,2)	← 1
P(2,·)	**A(2,2)** / B(2,0)	**A(2,0)** / B(0,1)	**A(2,1)** / B(1,2)	← 2
		↑1	↑2	

$T_{comm} = 2 \cdot \sqrt{P} \cdot \alpha + (n^2/\sqrt{P}) \cdot 2 \cdot \beta$

(This assumes a Mesh or Torus topology, where each processor has a direct physical link to its row and column neighbors.)

Algorithm & Communication

Cannon steps ($\sqrt{P}$ steps total):

1. Initial skew: shift A(i,·) left by i, B(·,j) up by j
2. Repeat $\sqrt{P}$ times:
   - Do local multiply: $C(i,j) \mathrel{+}= A_{local} \times B_{local}$
   - Shift A left by 1 (cyclic along row)
   - Shift B up by 1 (cyclic along col)

Communication per processor:

$$w = 2\sqrt{P} \cdot (n^2/P) = 2n^2/\sqrt{P}$$

The same bandwidth as SUMMA, but **no broadcast needed**

<u>Caveat</u>: **It has more assumptions: A** and **B** must be square and **P** must be a perfect square

Drawing made by Claude AI

Can We Do Better MatMul?

- Lower bound assumed 1 copy of data: $M = O(n^2/P)$ per proc
- But what if matrix small enough to fit $c > 1$ copies, so $M = cn^2/P$?
  - #words_moved = $\Omega(\text{\#flops} / M^{1/2}) = \Omega(n^2 / (c^{1/2} P^{1/2}))$
  - #messages = $\Omega(\text{\#flops} / M^{3/2}) = \Omega(P^{1/2} / c^{3/2})$
- Can we attain new lower bound?
  - A special case: "3D Matmul": $c = P^{1/3}$
    - Bernsten 89, Agarwal, Chandra, Snir 90, Aggarwal 95
    - Processors arranged in $P^{1/3} \times P^{1/3} \times P^{1/3}$ grid
    - Processor (i, j, k) performs C(i, j) = C(i, j) + A(i, k)*B(k, j), where each submatrix is $n/P^{1/3} \times n/P^{1/3}$
  - But not always that much memory available!

2.5D Matrix Multiply (1/2): Key Idea & Memory Trade-off

From 2D to 2.5D: Using Extra Memory

2D SUMMA baseline (c = 1 layer):
P procs on $\sqrt{P} \times \sqrt{P}$ grid → $W = \Theta(n^2/\sqrt{P})$

↓ In 2.5D algorithm, I can replicate data across **c** layers

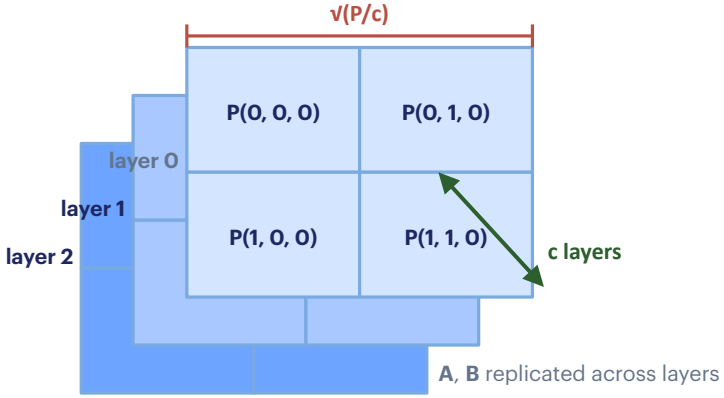
2.5D (c = n layers):

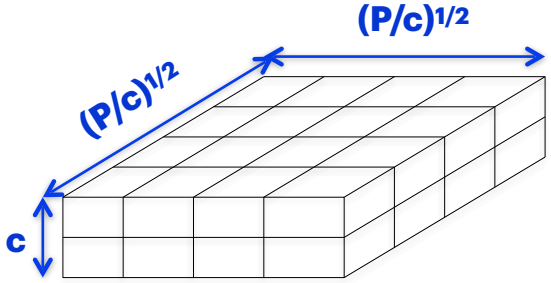
- I arrange P procs as $(\sqrt{(P/\mathbf{c})}) \times (\sqrt{(P/\mathbf{c})}) \times \mathbf{c}$
- Then, each of **c** layers holds a full copy of **C**
- **A** and **B** replicated **c** times across layers

The memory per processor: $\mathbf{M = c \cdot n^2/P}$ (**c** times more than 2D)
The **max useful c** is $\leq P^{1/3}$ to keep memory per processor $\leq O(n^2/P^{2/3})$

The Memory–Communication Trade-off:
c times more memory → $\sqrt{c}$ times **less communication**

Processor Layout: $(\sqrt{(P/c)}) \times (\sqrt{(P/c)}) \times c$





E.g.,: P = 32, c = 2

Drawing (top right) made by Claude AI

2.5D Matrix Multiply (2/2): Algorithm & α-β Cost

Algorithm

1. **Replicate A and B**
   - Broadcast **A** and **B** to all **c** layers
2. **Run SUMMA within each layer**
   - Each layer independently does 2D SUMMA on its (√(P/**c**)) × (√(P/**c**)) sub-grid nd each computes a partial **C**
3. **Reduce across layers**
   - All-reduce (sum) partial **C** tiles across the **c** layers for each (i,j) position

Each layer does √**c**× less work → √**c** times fewer SUMMA stages

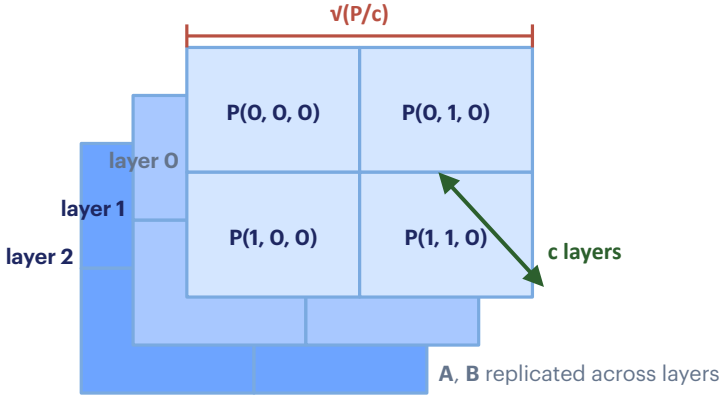
SUMMA in each layer (√(P/c) stages):

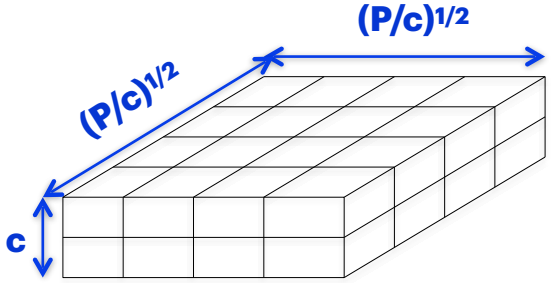
Dominant: $T_{comm} \approx \alpha \cdot \sqrt{(P/c)} \cdot \log(P/c) + \beta \cdot n^2/\sqrt{(Pc)} \cdot \log(P/c)$

(The cost of broadcast and allreduce are absorbed.)

It achieves 3D lower bound when $c = P^{1/3}$

Processor Layout: (√(P/c)) × (√(P/c)) × c



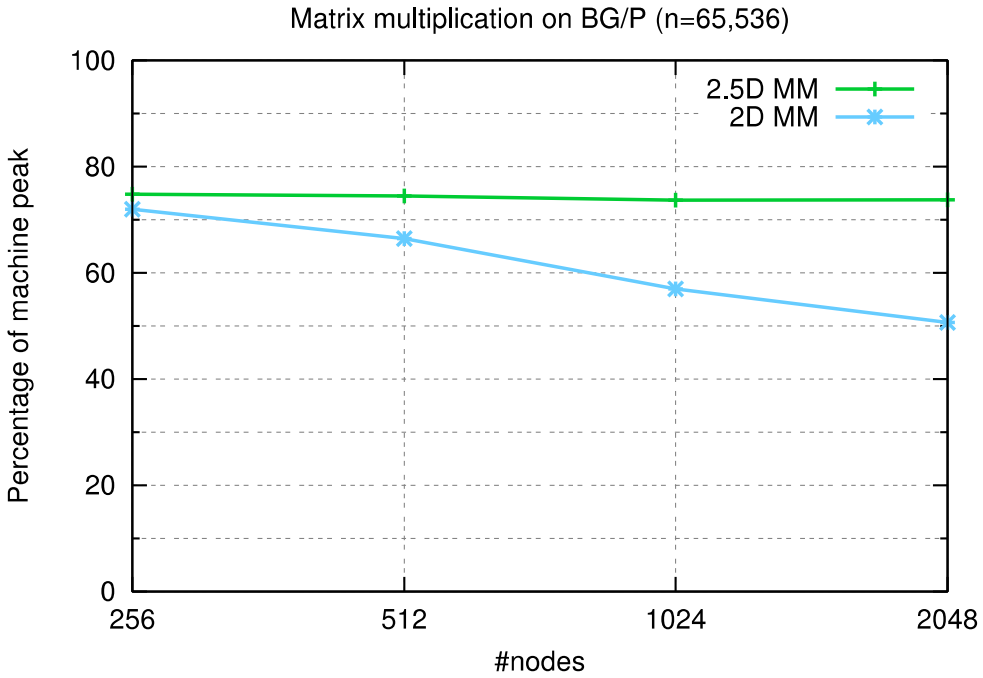


E.g.,: P = 32, c = 2

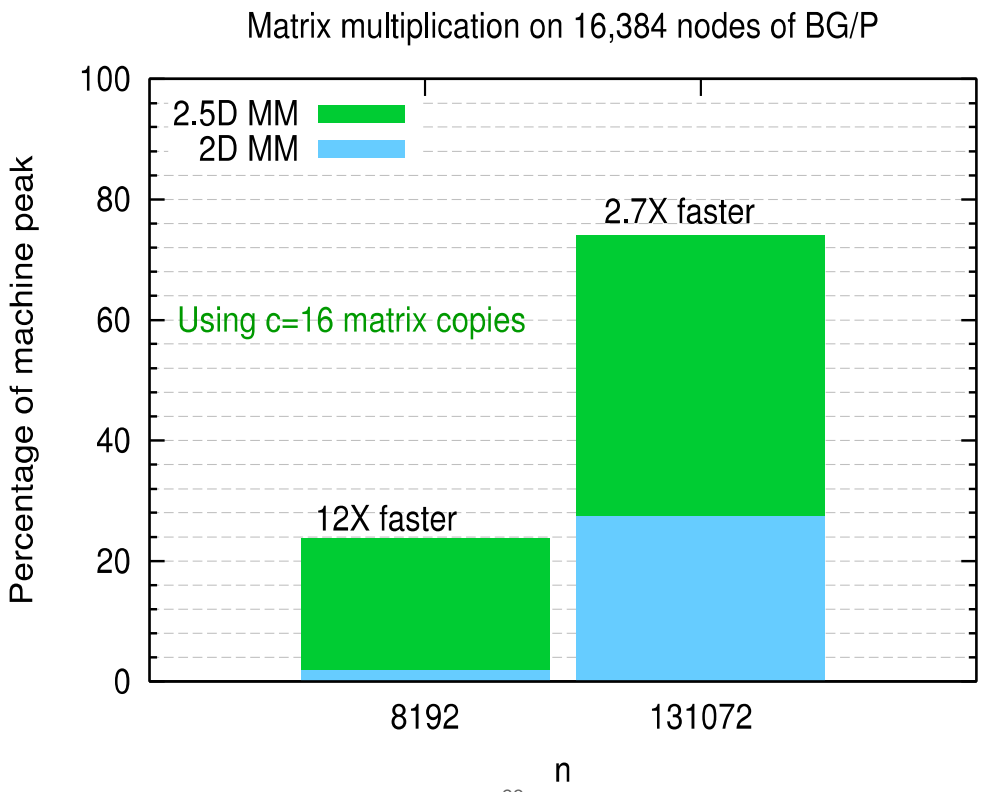
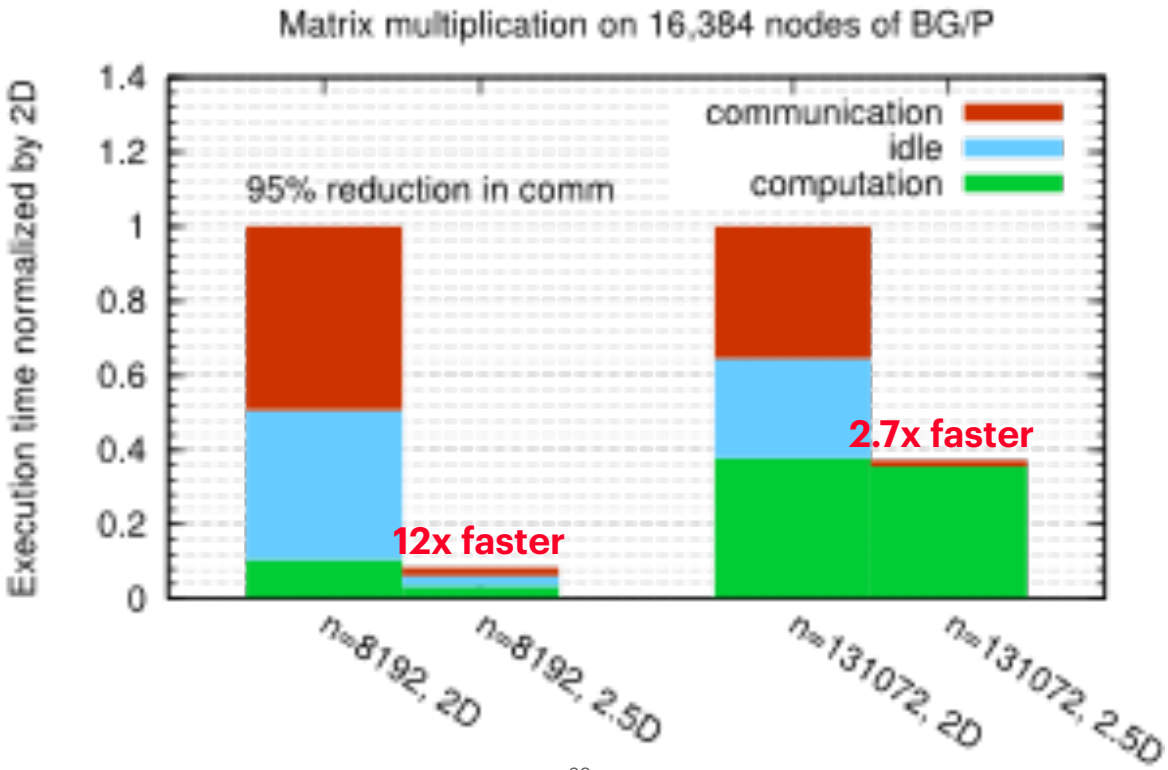
Drawing (top right) made by Claude AI

2.5D Matmul on IBM BG/P, n = 64K

- As P increases, available memory grows ➔ c increases proportionally to P
- Perfect strong scaling! But only up to c = P1/3



2.5D Matmul on IBM BG/P, 16K nodes / 64K cores



2.5D Matmul on IBM BG/P, 16K nodes / 64K cores